

OBJECT ORIENTED PROGRAMMING WITH C++

Unit-I: Introduction to OOP and C++ Basics

Comprehensive Study Notes
HPU BCA Semester III

1. Introduction to OOP

1.1 Need for OOP

Object Oriented Programming (OOP) evolved to overcome the limitations of procedural programming languages like C.

Problems with Procedural Programming

Problem	Description
Data Insecurity	Data is freely accessible to all functions
Code Reusability	Difficult to reuse code
Maintenance	Hard to maintain large programs
Real-World Modeling	Does not model real-world entities well
Complexity	Becomes complex as program grows

Solutions Provided by OOP

Solution	Description
Data Hiding	Data is protected and can only be accessed through member functions
Reusability	Inheritance allows code reuse
Maintainability	Easy to maintain and modify
Real-World Modeling	Models real-world entities using classes and objects
Modularity	Programs are divided into classes

1.2 Object Oriented Approach

The object-oriented approach models the real world by representing entities as **objects** that contain both **data** and **functions**. Unlike procedural programming where data and functions are separate, OOP combines them into a single unit called a class.

1.3 Characteristics of OOP

Characteristic	Description
Objects	Basic run-time entities, instances of classes
Classes	Blueprints for creating objects
Inheritance	One class acquires properties of another class
Polymorphism	Ability to take more than one form
Encapsulation	Wrapping data and methods into a single unit
Abstraction	Hiding implementation details, showing only essential features

1.3.1 Objects

Objects are the basic run-time entities in an object-oriented system. They are instances of classes.

Real-World Examples: Student, Teacher, Car, Book, Bank Account

1.3.2 Classes

Classes are user-defined data types that act as blueprints for creating objects. A class encapsulates data and functions together.

```
class Student {  
private:  
    int rollNo;  
    string name;  
    float marks;  
public:
```

```
void display() {  
    cout << "Roll No: " << rollNo << endl;  
}  
};
```

1.3.3 Inheritance

Inheritance is the process by which one class acquires the properties and methods of another class. It supports code reusability.

```
// Base class  
class Animal {  
public:  
    void eat() {  
        cout << "Eating..." << endl;  
    }  
};  
  
// Derived class  
class Dog : public Animal {  
public:  
    void bark() {  
        cout << "Barking..." << endl;  
    }  
};
```

1.3.4 Polymorphism

Polymorphism means the ability to take more than one form. It allows the same function name to be used for different purposes.

Type	Description
Compile-Time (Static)	Function overloading, Operator overloading
Run-Time (Dynamic)	Virtual functions, Function overriding

1.3.5 Encapsulation

Encapsulation is the wrapping up of data and methods into a single unit (class). It also involves hiding data from direct access using access specifiers like private, protected, and

public.

1.3.6 Abstraction

Abstraction is the process of hiding implementation details and showing only essential features. For example, when using `cout`, we only need to know `cout << data`, not how it internally handles the output.

1.4 Advantages of OOP

Advantage	Description
Reusability	Code can be reused through inheritance
Data Security	Data is protected through encapsulation
Modularity	Programs are divided into classes
Easy Maintenance	Easy to modify and maintain
Real-World Modeling	Models real-world entities effectively
Extensibility	Easy to extend functionality
Reduced Complexity	Manages complex systems effectively

1.5 Relationship between C and C++

C++ is a Superset of C:

- All C programs are valid C++ programs
- C++ adds OOP features to C
- C++ is more powerful and flexible
- C++ supports function overloading, references, classes, and exceptions

2. C++ Basics

2.1 Basic Program Construction

Structure of a C++ Program

```
+-----+
| Preprocessor Directives |
| #include <iostream>     |
+-----+
| Using Statement         |
| using namespace std;    |
+-----+
| Function Declaration     |
| return_type func(params);|
+-----+
| Main Function           |
| int main() { ... }      |
+-----+
| Function Definitions     |
| return_type func(params)|
| { ... }                 |
+-----+
```

Basic C++ Program - Hello World

```
// Preprocessor directive
#include <iostream>
using namespace std; // Using standard namespace

int main() // Main function - program entry point
{
    cout << "Hello World!" << endl;
    return 0;
}
```

Output:

```
Hello World!
```

2.2 Output Using cout

cout is used for output in C++. It uses the insertion operator `<<`.

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello World!" << endl;
    cout << "Number: " << 25 << endl;
    cout << "Sum: " << (10 + 20) << endl;
    cout << "C++ Programming";
    return 0;
}
```

Output:

```
Hello World!
Number: 25
Sum: 30
C++ Programming
```

Escape Sequences

Escape Sequence	Description
<code>\n</code>	Newline
<code>\t</code>	Tab
<code>\r</code>	Carriage Return
<code>\"</code>	Double Quote
<code>\\</code>	Backslash

<code>\0</code>	Null Character
-----------------	----------------

2.3 Preprocessor Directive

Preprocessor directives start with `#` and are processed before compilation.

```
#include <iostream>    // Includes header file for I/O
#include <cmath>        // Includes math functions
#define PI 3.14159     // Defines a macro constant
#define MAX(a,b) ((a)>(b)?(a):(b)) // Macro function

int main()
{
    cout << "Value of PI: " << PI << endl;
    cout << "Max of 10 and 20: " << MAX(10, 20) << endl;
    return 0;
}
```

2.4 Comments in C++

Comments are used to explain code and are ignored by the compiler.

```
// This is a single-line comment

/* This is a
   multi-line comment */

int main() {
    // This comment explains the next line
    cout << "Hello";
    return 0;
}
```

2.5 Integer Variables

```
int main()
{
    int a = 10;    // Positive integer
}
```



```
int b = -20;      // Negative integer
int c = 0;        // Zero
short int d = 100; // Short integer
long int e = 100000L; // Long integer

cout << a << " " << b << " " << c << " " << d;
return 0;
}
```

Output:

```
10 -20 0 100
```

Integer Modifiers

Modifier	Description	Range
short	Short integer	-32768 to 32767
long	Long integer	-2147483648 to 2147483647
signed	Signed integer	Both positive and negative
unsigned	Unsigned integer	Only positive

2.6 Character Variables

```
int main()
{
    char ch = 'A';
    char grade = 'B';

    cout << "Character: " << ch << endl;
    cout << "Grade: " << grade << endl;
    cout << "ASCII value of A: " << (int)ch << endl;
    return 0;
}
```

Output:

Character: A
Grade: B
ASCII value of A: 65

2.7 Input with cin

cin is used for input in C++. It uses the extraction operator `>>`.

```
#include <iostream>
using namespace std;

int main()
{
    string name;
    int age;

    cout << "Enter your name: ";
    cin >> name;
    cout << "Enter your age: ";
    cin >> age;

    cout << "Hello " << name << ", you are "
         << age << " years old." << endl;
    return 0;
}
```

Input: Rahul 22

Output:

```
Enter your name: Rahul
Enter your age: 22
Hello Rahul, you are 22 years old.
```

Multiple Inputs:

```
int a, b, c;
cin >> a >> b >> c; // Input: 10 20 30
```

2.8 Type bool

bool is used for boolean values: `true` (1) and `false` (0).

```
int main()
{
    bool isPassed = true;
    bool isFailed = false;

    cout << "isPassed: " << isPassed << endl;
    cout << "isFailed: " << isFailed << endl;
    cout << boolalpha;
    cout << "isPassed: " << isPassed << endl;
    cout << "isFailed: " << isFailed << endl;
    return 0;
}
```

Output:

```
isPassed: 1
isFailed: 0
isPassed: true
isFailed: false
```

2.9 setw Manipulator

setw is used to set the field width for output.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    int a = 10, b = 100, c = 1000;

    cout << "Without setw:" << endl;
    cout << a << endl << b << endl << c << endl;

    cout << "With setw(10):" << endl;
    cout << setw(10) << a << endl;
```

```
    cout << setw(10) << b << endl;
    cout << setw(10) << c << endl;
    return 0;
}
```

Other Manipulators

Manipulator	Description
setw(n)	Set field width to n
setprecision(n)	Set precision to n
setfill(ch)	Set fill character
left	Left justify
right	Right justify
fixed	Fixed decimal point
scientific	Scientific notation

2.10 Type float

float is used for floating-point numbers.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    float pi = 3.14159f;
    float temp = 98.6f;
    float price = 199.99f;

    cout << "Pi: " << pi << endl;
    cout << "Temperature: " << temp << endl;
    cout << "Price: $" << price << endl;
    cout << fixed << setprecision(2);
    cout << "Pi (2 decimals): " << pi << endl;
}
```

```
    return 0;  
}
```

Output:

```
Pi: 3.14159  
Temperature: 98.6  
Price: $199.99  
Pi (2 decimals): 3.14
```

Floating-Point Types

Type	Size	Precision
float	4 bytes	7 decimal digits
double	8 bytes	15 decimal digits
long double	10-16 bytes	19 decimal digits

2.11 Type Conversion

Type conversion is the process of converting one data type to another.

```
int main()  
{  
    // Implicit conversion  
    int a = 10;  
    double b = a;          // int to double: b = 10.0  
  
    // Explicit conversion (Casting)  
    double c = 3.14;  
    int d = (int)c;        // double to int: d = 3  
  
    int e = int(c);        // Functional notation  
  
    // char to int  
    char ch = 'A';  
    int ascii = (int)ch;   // ascii = 65  
  
    cout << "b: " << b << endl;
```

```
    cout << "d: " << d << endl;  
    cout << "ASCII of 'A': " << ascii << endl;  
    return 0;  
}
```

Output:

```
b: 10  
d: 3  
e: 3  
f: 3  
ASCII of 'A': 65
```

3. Operators in C++

3.1 Arithmetic Operators

Operator	Description	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (Remainder)	a % b
++	Increment	a++
--	Decrement	a--

```
int main()
{
    int a = 10, b = 3;
    cout << "a + b = " << (a + b) << endl;
    cout << "a - b = " << (a - b) << endl;
    cout << "a * b = " << (a * b) << endl;
    cout << "a / b = " << (a / b) << endl;
    cout << "a % b = " << (a % b) << endl;

    double c = 10.0, d = 3.0;
    cout << "c / d = " << (c / d) << endl;
    return 0;
}
```

Output:

```
a + b = 13
a - b = 7
a * b = 30
```

```
a / b = 3
a % b = 1
c / d = 3.33333
```

3.2 Relational Operators

Operator	Description	Example
==	Equal to	a == b
!=	Not equal to	a != b
<	Less than	a < b
>	Greater than	a > b
<=	Less than or equal to	a <= b
>=	Greater than or equal to	a >= b

```
int main()
{
    int a = 10, b = 20;

    cout << boolalpha;
    cout << "a == b: " << (a == b) << endl;
    cout << "a != b: " << (a != b) << endl;
    cout << "a < b: " << (a < b) << endl;
    cout << "a > b: " << (a > b) << endl;
    cout << "a <= b: " << (a <= b) << endl;
    cout << "a >= b: " << (a >= b) << endl;
    return 0;
}
```

Output:

```
a == b: false
a != b: true
a < b: true
a > b: false
```



```
a <= b: true  
a >= b: false
```

3.3 Logical Operators

Operator	Description	Example
&&	AND	a && b
	OR	a b
!	NOT	!a

```
int main()  
{  
    bool a = true, b = false;  
    int x = 10, y = 20, z = 30;  
  
    cout << boolalpha;  
    cout << "a && b: " << (a && b) << endl;  
    cout << "a || b: " << (a || b) << endl;  
    cout << "!a: " << (!a) << endl;  
    cout << "!b: " << (!b) << endl;  
    cout << "(x < y) && (y < z): "  
        << ((x < y) && (y < z)) << endl;  
    return 0;  
}
```

Output:

```
a && b: false  
a || b: true  
!a: false  
!b: true  
(x < y) && (y < z): true
```

3.4 Bitwise Operators

Operator	Description	Example
&	Bitwise AND	a & b
	Bitwise OR	a b
^	Bitwise XOR	a ^ b
~	Bitwise NOT	~a
<<	Left shift	a << n
>>	Right shift	a >> n

3.5 Assignment Operators

Operator	Description	Example	Equivalent to
=	Simple assignment	a = b	a = b
+=	Add and assign	a += b	a = a + b
-=	Subtract and assign	a -= b	a = a - b
*=	Multiply and assign	a *= b	a = a * b
/=	Divide and assign	a /= b	a = a / b
%=	Modulus and assign	a %= b	a = a % b

3.6 Ternary Operator (Conditional Operator)

```
// Syntax: condition ? expression1 : expression2
int max = (a > b) ? a : b; // Returns larger of a and b
```

3.7 sizeof Operator

```
cout << "Size of int: " << sizeof(int) << " bytes" << endl;  
cout << "Size of float: " << sizeof(float) << " bytes" << endl;
```

3.8 Scope Resolution Operator (::)

The **scope resolution operator** is used to access global variables when they are hidden by local variables.

```
int x = 100; // Global variable  
  
int main()  
{  
    int x = 50; // Local variable  
    cout << "Local x: " << x << endl;  
    cout << "Global x: " << ::x << endl;  
    return 0;  
}
```

Output:

```
Local x: 50  
Global x: 100
```

3.9 endl and setw

- endl - Inserts a newline and flushes the output buffer
- setw(n) - Sets the field width for the next output operation

3.10 I/O Operators

- << - Insertion operator (used with cout)
- >> - Extraction operator (used with cin)

4. Control Structures

4.1 if Statement

The **if statement** executes a block of code if the condition is true.

```
int main()
{
    int num;
    cout << "Enter a number: ";
    cin >> num;

    if (num > 0) {
        cout << num << " is positive." << endl;
    }
    return 0;
}
```

Output:

```
Enter a number: 5
5 is positive.
```

4.2 if-else Statement

The **if-else statement** executes one block if the condition is true and another if it is false.

```
int main()
{
    int num;
    cout << "Enter a number: ";
    cin >> num;

    if (num % 2 == 0) {
        cout << num << " is even." << endl;
    } else {
        cout << num << " is odd." << endl;
    }
}
```

```
    }  
    return 0;  
}
```

Output:

```
Enter a number: 7  
7 is odd.
```

4.3 Nested if

Nested if is an if statement inside another if statement.

```
int main()  
{  
    int num;  
    cout << "Enter a number: ";  
    cin >> num;  
  
    if (num >= 0) {  
        if (num == 0) {  
            cout << "Number is zero" << endl;  
        } else {  
            cout << "Number is positive" << endl;  
        }  
    } else {  
        cout << "Number is negative" << endl;  
    }  
    return 0;  
}
```

4.4 else-if Ladder

The **else-if ladder** is used to test multiple conditions.

```
int main()  
{  
    int marks;  
    cout << "Enter your marks: ";  
    cin >> marks;
```

```
if (marks >= 90) {  
    cout << "Grade: A+ (Excellent)" << endl;  
} else if (marks >= 75) {  
    cout << "Grade: A (Very Good)" << endl;  
} else if (marks >= 60) {  
    cout << "Grade: B (Good)" << endl;  
} else if (marks >= 45) {  
    cout << "Grade: C (Average)" << endl;  
} else {  
    cout << "Grade: F (Fail)" << endl;  
}  
return 0;  
}
```

Output:

```
Enter your marks: 85  
Grade: A (Very Good)
```

4.5 switch Statement

The **switch statement** is used for multi-way branching based on the value of an expression.

```
int main()  
{  
    char op;  
    double a, b;  
  
    cout << "Enter operator (+, -, *, /): ";  
    cin >> op;  
    cout << "Enter two numbers: ";  
    cin >> a >> b;  
  
    switch (op) {  
        case '+':  
            cout << a << " + " << b << " = " << (a + b);  
            break;  
        case '-':  
            cout << a << " - " << b << " = " << (a - b);  
            break;
```

```
        case '*':
            cout << a << " * " << b << " = " << (a * b);
            break;
        case '/':
            if (b != 0)
                cout << a << " / " << b << " = " << (a / b);
            else
                cout << "Division by zero!";
            break;
        default:
            cout << "Invalid operator!";
    }
    return 0;
}
```

Output:

```
Enter operator (+, -, *, /): *
Enter two numbers: 10 5
10 * 5 = 50
```

4.6 Conditional Operator (Ternary Operator)

The **conditional operator** is a shorthand for if-else.

```
int main()
{
    int a = 10, b = 20;
    int max = (a > b) ? a : b;
    cout << "Maximum: " << max << endl;

    int num = 5;
    cout << num << " is "
         << ((num % 2 == 0) ? "Even" : "Odd") << endl;
    return 0;
}
```

Output:

```
Maximum: 20
5 is Odd
```

5. Data Types, Tokens, and Variables

5.1 C++ Tokens

Tokens are the smallest individual units in a C++ program. Types include:

- **Keywords** - Reserved words (int, float, if, else, while, class, etc.)
- **Identifiers** - Names given to variables, functions, classes
- **Constants** - Fixed values (literal constants)
- **Strings** - Sequence of characters enclosed in double quotes
- **Operators** - Symbols that perform operations

5.2 Data Types in C++

Basic Data Types

Type	Size	Range / Values
int	4 bytes	-2,147,483,648 to 2,147,483,647
char	1 byte	-128 to 127 or 0 to 255
float	4 bytes	3.4e-38 to 3.4e+38
double	8 bytes	1.7e-308 to 1.7e+308
void	0 bytes	No value
bool	1 byte	true / false

Derived Data Types

- Arrays
- Functions
- Pointers
- References

User-Defined Data Types

- Structures (struct)
- Classes (class)
- Unions (union)
- Enumerations (enum)

5.3 Variables

Variable Declaration:

```
int age;           // Declaration
float salary;      // Declaration
char grade;        // Declaration
```

Variable Initialization:

```
int age = 20;           // C-style initialization
int count(10);          // Constructor-style
int value{100};          // Uniform initialization (C++11)
```

Variable Scope:

- **Local variables** - Declared inside a function or block
- **Global variables** - Declared outside all functions

6. Questions for Practice

Short Answer Questions (2-5 Marks)

1. What is OOP? Why is it needed?
2. What are the characteristics of OOP?
3. What is a class? What is an object?
4. What is inheritance?
5. What is polymorphism?
6. What is encapsulation?
7. What is the difference between C and C++?
8. What is `cout` ? How is it used?
9. What is `cin` ? How is it used?
10. What is the use of `setw` manipulator?

Long Answer Questions (10 Marks)

1. Explain the need for OOP and its characteristics.
2. Write a C++ program to accept and display student details.
3. Explain the basic structure of a C++ program.
4. Explain different data types in C++.
5. Explain type conversion in C++ with examples.

Program Writing Questions (5 Marks)

1. Write a program to swap two numbers.
2. Write a program to calculate the area of a rectangle.
3. Write a program to accept two numbers and display their sum, difference, product, and quotient.

Objective Questions (1 Mark)

1. C++ is a:
(a) Procedural language (b) OOP language (c) Both (d) None

2. `cout` is used for:
(a) Input (b) Output (c) Both (d) None
3. `cin` is used for:
(a) Input (b) Output (c) Both (d) None
4. The default namespace in C++ is:
(a) `std` (b) `cpp` (c) `iostream` (d) None

Quick Revision

OOP Characteristics Summary

Characteristic	Definition
Objects	Instances of classes - runtime entities
Classes	Blueprints for creating objects
Inheritance	Acquiring properties of parent class
Polymorphism	Same name, multiple forms (overloading/overriding)
Encapsulation	Wrapping data + functions into a single unit
Abstraction	Hiding implementation details

C++ Program Structure

```
#include <iostream>
using namespace std;

int main()
{
    // Body of the program
    cout << "Hello World!" << endl;
    return 0;
}
```

Data Types Summary

Type	Size	Example
int	4 bytes	10

char	1 byte	'A'
float	4 bytes	3.14f
double	8 bytes	3.14159
bool	1 byte	true/false

End of Unit-I Notes